

Lecture 15

FAIR in Practice

Interoperability · Reusability · Reproducibility



Guest contribution: Jamie McCann introduced PlantGenIE live in Lecture 14

In these exercises you will query real research infrastructure from the command line — demonstrating FAIR in your own institution's primary research data.

Full written guide, commands & background: hpc2n.github.io/bioinformatics-hpc → 15. FAIR in Practice (also linked from the course GitHub repo)

Session Aims & ILO Links

- | | | |
|---|---|-----------|
| 1 | Query a REST API from the command line and explain how APIs (Application Programming Interfaces) embody the Interoperable principle | ILO 1, 3 |
| 2 | Explain PlantGenIE as an example of research infrastructure designed for interoperability | ILO 3 |
| 3 | Apply a metadata sufficiency checklist to a real GEO/ENA dataset | ILO 3, 13 |
| 4 | Reconstruct a bioinformatics analysis pipeline from a methods section and identify reproducibility gaps | ILO 3, 13 |
| 5 | Commit all exercise outputs, commands, and findings to Git with structured documentation | ILO 12 |
| 6 | Apply the FAIR essay marking criteria to your own preliminary paper assessment | ILO 13 |

These are self-study exercises. Bring questions from Lecture 14 to office hours or the Canvas discussion board. The essay assessment closes [date — see Canvas].

FAIR Applied: Exercise Overview

Each exercise maps directly to a FAIR principle — and to the essay assessment.

I — Interoperable



PlantGenIE API

Query the PlantGenIE REST (Representational State Transfer) API from the command line. Retrieve gene information, expression data, and BLAST results in standard formats. Understand why API design choices either enable or limit FAIR compliance.

*Builds on Jamie McCann's
Lecture 14 introduction*

R — Reusable



GEO/ENA Metadata Assessment

Apply the metadata sufficiency checklist to a real dataset. Determine what is present, what is missing, and what the practical consequences of each gap would be for a researcher attempting reanalysis.

Independent exercise

R — Reusable



Paper Reproducibility Exercise

Reconstruct the analysis pipeline from your chosen essay paper. Build the evidence base you will use in the essay. Identify the most significant reproducibility gaps.

*Independent exercise using
your essay paper*

Part 1

I = Interoperable: The PlantGenIE API

PlantGenIE: Research Infrastructure at Umeå

 Developed at Umeå University · plantgenie.se · github.com/plantgenie



Genome browsers

Reference genomes for *Picea abies* (Norway spruce) and *Populus tremula* (European aspen), with gene models, annotations, and synteny tools



Expression data

Integrated RNA-seq expression atlases across tissues and conditions, with co-expression network tools for identifying gene modules



BLAST and search

Sequence similarity search against conifer and aspen genomes — accessible via web interface and API



REST API

All data accessible programmatically via documented HTTP endpoints returning JSON and FASTA. You will use this throughout these exercises.

Jamie McCann introduced PlantGenIE and its API design decisions live in Lecture 14 — the exercises below put that into practice.

Why APIs Embody the Interoperable Principle

A well-designed API means: any tool, any programming language, any researcher — access the same data in the same way.

11

Uses a formal, accessible, broadly applicable language

REST over HTTPS — the same protocol every web browser uses. Any tool that can make HTTP requests can query the API.

12

Uses vocabularies that follow FAIR principles

Gene entries include GO term identifiers (GO:XXXXXXX), InterPro accessions, NCBI taxonomy IDs — stable, machine-readable ontology terms, not free text.

13

Includes qualified references to other data

Gene entries cross-reference ENA accessions, Ensembl gene IDs, UniProt accessions — forming a web of connected resources.

+

Documented endpoints

The API documentation describes what each endpoint accepts and returns. A researcher can query it without contacting the authors or using a specific browser.

Compare: a database that only provides data via a browser download, with no stable URL and no documented structure, is not Interoperable — even if the data is technically available.

Exercise 1: Querying the PlantGenIE API

Login to Kebnekaise via OnDemand. Full commands in your handout.



Setup

Create lecture15-fair-practice directory · git init · initial commit



1A — Gene
info

Query gene information for a *Populus tremula* gene (POST request) · inspect JSON (JavaScript Object Notation) response · extract GO terms and cross-references



1B —
Expression

Retrieve expression data for the AspWood experiment · identify the most highly expressed sample · note whether expression units are specified



1C — BLAST
via PlantGenIE

PlantGenIE's BLAST endpoint is confirmed working — commands on the next slide. The three ways to run BLAST (1C–1F) follow after that.



Commit your
progress

Document 1A/1B findings in README · git add · git commit — you'll add BLAST findings and a final summary commit in 1F

If plantgenie.se is unavailable, check the current API documentation or post on the Canvas discussion board.

GET vs POST — Two Ways to Ask a Server

You will use both today — knowing which one an API expects is essential.

GET

- Asks the server to return something
- Parameters go in the URL itself
- curl's default — no -X flag needed
- What you used for NCBI/UniProt in Lecture 11

POST

- Sends data to the server as part of the request
- Data goes in the request body (curl -d), not the URL
- Marked explicitly: curl -X POST
- PlantGenIE uses this even to ask for data — the query itself (a list of gene IDs) is too structured to fit in a URL

Both are just HTTP conventions — the server decides what each endpoint actually does with a given method, so always check the documentation rather than assume.

Exercise 1: Key API Commands

Parts 1A and 1B — gene information and expression

```
$ BASE="https://www.plantgenie.se/api"
$ SPECIES="populus-tremula"; GENE_ID="Potra2n18c32336"
# 1A: Retrieve gene info (POST with JSON body)
$ curl -s -X POST "${BASE}/v1/annotations" \
$   -d "{\"species\":\"${SPECIES}\",\"geneIds\": [\"${GENE_ID}\"]}" > gene_info.json
# 1B: Retrieve expression data (AspWood experiment, ERP016242)
$ curl -s -X POST "${BASE}/v1/expression" \
$   -d "{\"experimentId\":\"15\",\"geneIds\": [\"${GENE_ID}\"]}" > expression_data.json
$ cat gene_info.json expression_data.json
```

Part 1C — BLAST via PlantGenIE (confirmed working)

```
$ curl -s -X POST "${BASE}/v1/blast/blastn/submit?database_type=cds" \\
$   -F "species_id=3" -F "genome_id=3" -F "file=@seq.fasta"
# Returns {"jobId": "..."} – poll until status is "SUCCESS"
$ curl -s "${BASE}/v1/blast/poll/${JOB_ID}"
$ curl -s "${BASE}/v1/blast/retrieve/${JOB_ID}/tsv"
# Use www.plantgenie.se – BLAST is broken on dev.plantgenie.se

$ git add . && git commit -m "lecture15: PlantGenIE API exercises complete"
```

Exercise 1 continued — Three Ways to Run BLAST

Same TP53 sequence, three tools, one workflow. Full commands in your handout.

1C — Web BLAST

Run `blastp` via NCBI (blast.ncbi.nlm.nih.gov) or PlantGenIE (plantgenie.se) - note the parameters used and how long the search takes

1D — Local HPC

Submit the same search as a Slurm job on Kebnekaise against the local Swiss-Prot database - compare setup effort to the website

1E — API

Submit via the EBI BLAST API (submit, poll, retrieve) - under 30s to complete; NCBI's equivalent works too but has no turnaround guarantee

1F — Wrap Up

Complete the comparison table (speed, scalability, reproducibility, control) in your README and commit your findings

Reflect

Which approach would you choose for 1 sequence versus 500? Why does that change depending on whether reproducibility matters?

Background on how BLAST works (seeding, E-values, scoring) is in the Lecture 12 reference material - read it before starting this part if you have not already.

Part 2

R = Reusable: Metadata assessment and paper reproducibility

The Reusability Challenge

Deposited data is not the same as reusable data.

Scenario: not reusable

A GEO entry has 12 RNA-seq samples. Raw FASTQ files are available. The samples are labelled "sample_01" through "sample_12". The GEO description says "expression profiling of treated vs untreated cells." There are no ontology terms, no specification of cell type, no indication of which samples are treated and which are control, and the reference genome version is not stated.

Result: a researcher who finds this dataset cannot determine experimental conditions without contacting the authors. The data exists, but it is not reusable.

Scenario: reusable

The same 12 samples, but each is annotated with: organism (NCBI:txid9606), tissue (UBERON:0002368), treatment (EFO:0001461 for dexamethasone treatment), concentration and duration, and a link to the reference genome accession (GCF_000001405.40). Each condition has 3 biological replicates clearly labelled. Analysis code is available with a Zenodo DOI.

Result: an independent researcher can immediately reproduce the differential expression analysis from the raw data alone.

The difference is not the data — it is the metadata. This is exactly what your exercise assesses.

Exercise 2: The Metadata Sufficiency Checklist

Dataset: ERP016242 (an ENA Study accession — see Lecture 11) · Sundell et al. (2017) AspWood · *Populus tremula* wood formation · The Plant Cell

Sample identification

- ☐ Findable by search without knowing accession in advance?
- ☐ All samples identifiable and assignable to experimental conditions?
- ☐ Biological replicates clearly labelled?

Biological metadata

- ☐ Organism with NCBI taxonomy ID?
- ☐ Tissue/cell type with UBERON or CL ontology term?
- ☐ Treatment with EFO or CHEBI ontology term?
- ☐ Other variables (age, sex, genotype, time point)?

Exercise 2, Part 2

Dataset: ERP016242 · Sundell et al. (2017) AspWood · *Populus tremula* wood formation · The Plant Cell

Technical metadata

- ☐ Sequencing platform specified?
- ☐ Library type (stranded, poly-A/ribo-depl.) specified?
- ☐ Raw FASTQ files available (not only processed data)?
- ☐ Reference genome/annotation version specified?

Reproducibility

- ☐ Analysis code available? Is it versioned?
- ☐ All software versions specified?
- ☐ Community metadata standard declared (MINSEQE)?

Exercise 3: Paper Reproducibility Assessment

Open your chosen essay paper. Work through the following systematically.

1

Map the analysis pipeline

List every computational step in the methods. For each: tool, version specified?, parameters reported? Build a table — see example below.

2

Assess code availability

Is code available at all? If yes: is it versioned (Git tag/Zenodo), documented, with dependencies pinned? If no: is there any reproducibility path?

3

Assess the reference genome

Is the reference genome specified? Is the assembly accession version given? Is the annotation GTF version specified? Could you identify exactly the right reference to reproduce the alignment?

4

Write a reproducibility verdict

2-3 sentences: could an independent researcher reproduce the main analysis? What specific information would they need from the authors that is not in the paper?

Example pipeline table in your handout. This becomes the evidence base for your essay — take careful notes.

Example: Reconstructing the Analysis Pipeline

Build this table from the methods section of your chosen paper. This is your FAIR evidence base.

Step	Tool	Version?	Parameters?	Notes
Quality trimming	Trimmomatic	v0.39 ✓	LEADING:3 TRAILING:3 MINLEN:36 ✓	Well reported
Alignment	HISAT2	v2.1.0 ✓	'default parameters' ✗	Parameters missing
Quantification	featureCounts	Not specified ✗	Not specified ✗	Version + params missing
DE analysis	DESeq2	v1.20.0 ✓	Shrinkage method not stated ✗	Partial — key parameter absent
Reference genome	GRCh38	Release not specified ✗	—	Cannot confirm exact version

'Default parameters' is almost never sufficient for reproducibility — defaults change between software versions, and many tools have multiple default modes depending on other settings. This is one of the most common reproducibility failures in bioinformatics methods sections.

Part 4 — Building a Reproducible Report

Turn a small piece of this exercise into a rerunnable notebook. Full commands in your handout.

 **Build the notebook**

Launch Jupyter via OnDemand · create 3 cells: a markdown description, code that loads and summarises `expression_data.json`, and a markdown interpretation

 **Restart & Run All**

Kernel → Restart & Run All — confirms the notebook reproduces its own output from scratch, not just that it ran once while you were writing it

 **Export & commit**

Export to HTML (File → Download as → HTML) · `git add` · commit both the notebook and its HTML export

 **The R equivalent**

Same idea in R uses R Markdown + knitr via RStudio Server — you'll use this properly in Data Science for Biology with R. Not required for this exercise.

 **Reflect**

What did the notebook make explicit that a bare script or terminal history would not? Could a labmate reproduce your output from the `.ipynb` file alone?

See Lecture 14, Section 4.5 for why notebooks and R Markdown matter for reproducibility.

Exam Prep: Pushing Your Work to GitHub

You have run git init/add/commit all lecture - the exam also requires pushing to a remote.

Before the Exam

- Create a personal GitHub account now if you don't have one - free, and used throughout the programme
- Send Nathaniel your GitHub username before the exam - your private exam repo is created in advance from it
- Watch for a Canvas announcement with the exact deadline, closer to the exam date

Practice It Now

Push lecture15-fair-practice to your own GitHub:

```
git remote add origin \
  git@github.com:<user>/lecture15-fair-
  practice.git
git branch -M main
git push -u origin main
```

Same repository the exam checks - its push history, not local timestamps.

Full explanation of remotes: Lecture 7, “Adding a Remote Repository” and “Uploading Changes”.

Essay Assessment

From these exercises to the Canvas assignment — full assessment details in Lecture 14, Section 10

From Exercise to Essay: The Direct Connection

Everything you do in these exercises is direct preparation for the essay.

Exercise: PlantGenIE API exercise

→ Essay: Evidence for the Interoperable principle — you have now experienced what a well-designed API enables. Use this as a reference point when assessing I in your essay paper.

Exercise: GEO metadata checklist

→ Essay: Direct template for your Reusability assessment. Apply the same checklist to your essay paper's dataset. Your completed checklist is your methodology.

Exercise: Pipeline reconstruction table

→ Essay: This IS your reproducibility evidence. The table you build here becomes the backbone of your essay's computational reproducibility section.

Exercise: Reproducibility verdict

→ Essay: The 2-3 sentence verdict you write here is the draft of your essay's synthesis paragraph. Develop it with specific consequences and recommendations.

What You Should Know After These Exercises



Query a REST API from the command line and parse JSON responses with grep



Explain how APIs embody FAIR Interoperability — standard protocols, ontology terms, cross-references, documented endpoints



Describe PlantGenIE as research infrastructure designed for programmatic access to boreal tree genomics data



Apply the metadata sufficiency checklist to a GEO/ENA dataset and identify specific reusability gaps



Reconstruct a bioinformatics analysis pipeline from a methods section, noting version and parameter gaps



Write a reproducibility verdict identifying what is missing and what it would prevent



Commit all exercise outputs and findings to Git with structured README documentation



Describe the G/VG distinction: description vs analysis-of-consequence-with-recommendation

Essay deadline: see Canvas. Bring questions to office hours or post on the Canvas discussion board.